MODEL VALIDATION · ENSEMBLE

모델 검증과 앙상블

모델의 성능을 공정하게 재고, 튜닝으로 높이고, 여러 모델을 묶는 방법

Cross-Validation, Hyperparameter Tuning and Ensemble Learning

모델이 모르는 데이터에서 성능을 어떻게 재는가, 그 성능을 끌어올리는 하이퍼파라미터를 어떻게 찾는가,
여러 모델을 어떻게 묶어 단일 모델보다 나은 예측을 얻는가를 차례로 다룬다.

검증으로 성능을 재고, 튜닝으로 높이고, 앙상블로 결합한다

교차검증 점수가 튜닝의 기준이 되고, 튜닝한 모델들을 묶어 앙상블을 만든다

1

교차 검증 (Cross-Validation)

데이터를 나누는 여러 방식과 층화 K겹을 보고, 시계열에서 정보가 새지 않게 검증한다

2

파라미터 튜닝 (Hyperparameter Tuning)

그리드 서치와 랜덤 서치를 거쳐 베이지안 최적화로 하이퍼파라미터를 찾는다

3

앙상블 (Ensemble)

배깅과 부스팅의 구조를 보고, 투표 앙상블과 평균 블렌딩을 실습한다

4. 스태킹 — 예측을 입력으로 받는 메타 모델을 훈련하고, CV로 정보 누수를 막는다

01

PART 1 · 교차 검증

교차 검증

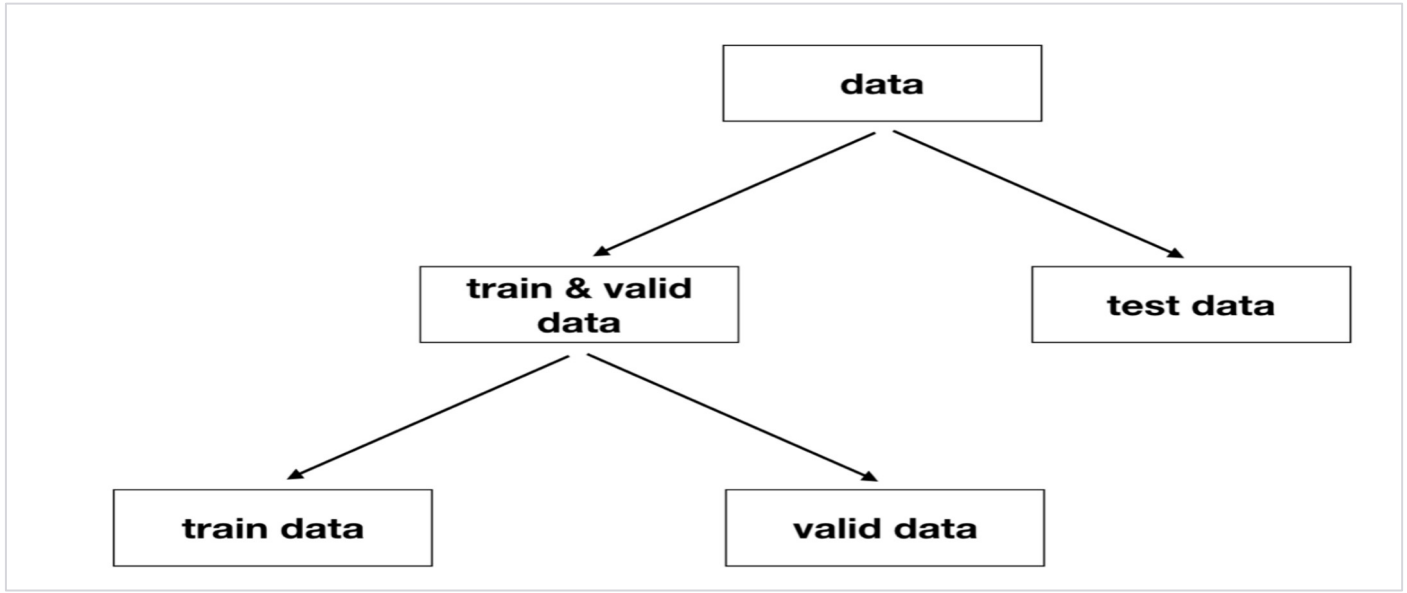
모델이 모르는 데이터에서 성능을 공정하게 재는 방법

데이터는 훈련·검증·테스트의 세 역할로 나눈다

훈련·검증 데이터(train & valid)는 다시 둘로 나뉘고, 테스트 데이터는 끝까지 떼어 둔다

세 데이터의 역할

- 훈련 데이터: 모델을 학습하는 데 쓰는 데이터
- 검증 데이터: 학습한 모델의 성능을 검증하는 데이터. 모델이 모르는 데이터로, 하이퍼파라미터를 고르는 데 쓴다
- 테스트 데이터: 고른 하이퍼파라미터로 학습한 모델이 예측하는 데이터. 역시 모델이 모르는 데이터다

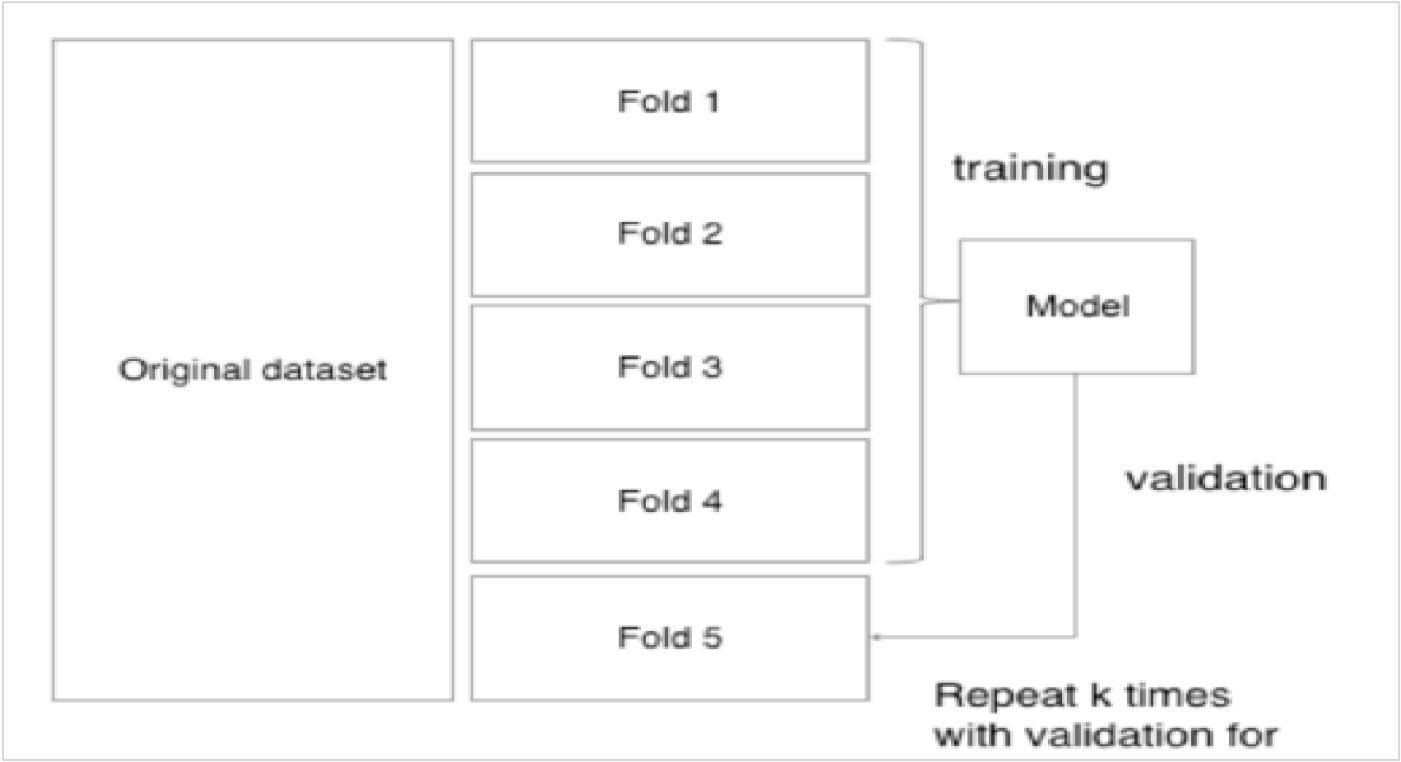


K겹 교차검증에서는 모든 겹이 한 번씩 검증 데이터가 된다

K-fold: 데이터를 K개로 나눠 K−1개로 학습하고 나머지 1개로 검증한다

K겹(K-fold)의 작동

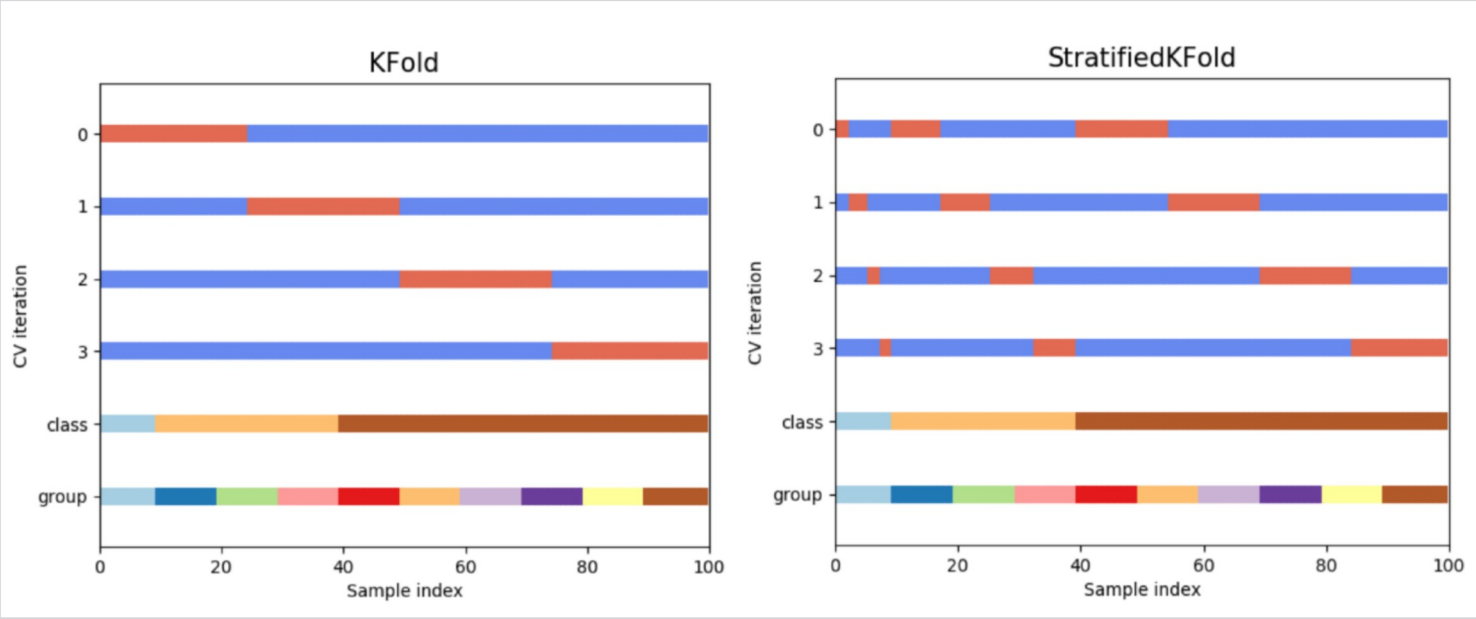
- 데이터를 K개의 겹(fold)으로 분할한다
- K−1개 겹으로 모델을 학습하고 나머지 1개 겹으로 검증한다
- 검증 겹을 바꿔 가며 K번 반복하므로, K번 서로 다른 데이터셋으로 학습한 모델을 검증할 수 있다



층화 K겹은 각 겹의 클래스 비율을 전체와 같게 맞춘다

stratify의 의미 — 위: KFold, 아래: StratifiedKFold (빨강 = 검증 구간, class = 클래스 분포)

- K겹은 데이터의 정렬 여부와 클래스 비율에 상관없이 분할한다. 한 겹에 특정 클래스가 몰리면 그 겹은 학습 데이터셋을 대표하지 못한다
- 층화 K겹(stratified k-fold)은 K개로 나눈 뒤에도 각 겹의 클래스 비율을 전체 훈련 데이터셋의 클래스 비율과 같게 맞춘다



정렬된 데이터에 KFold를 쓰면 검증 겹에 한 클래스만 남는다

KFold(n_splits=5) — 빨간 상자가 각 겹의 검증 레이블이다

[입력] 프롬프트 · Claude Code (Code 탭)

```
iris 데이터를 레이블 순서(0→1→2) 그대로 써서 KFold를 시험해 줘.
1. KFold(n_splits=5)로 섞지 말고 나눠 줘.
2. 겹마다 훈련·검증 레이블을 출력해 줘.
3. 검증 겹별 클래스 개수를 표로 정리해 줘.
4. 현재 sklearn API로 써 줘.
```



검토할 것

- 0·2·4겹의 검증 레이블이 한 클래스뿐인가?
- 겹별 개수의 합이 30인가?
- shuffle 없이 random_state를 넘기지 않았는가?

출력 읽기

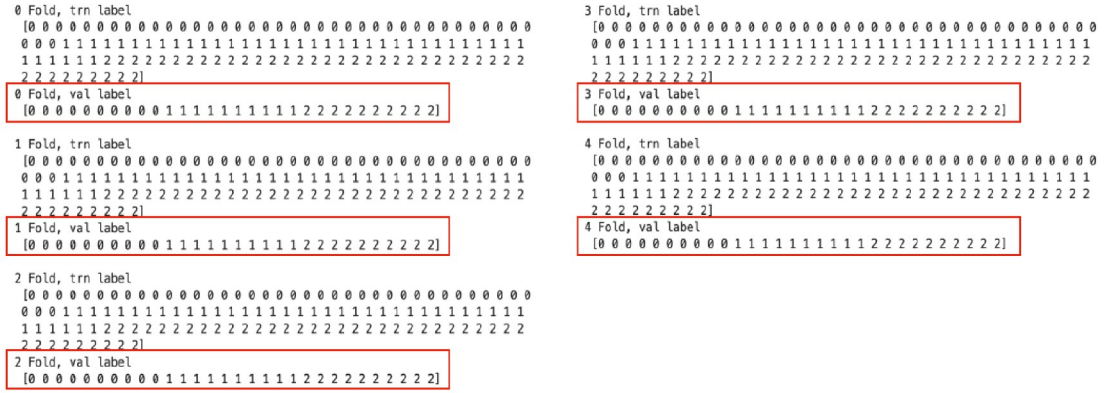
- 레이블이 0→1→2 순으로 정렬돼 있다
- 0겹 검증 레이블은 모두 0, 2겹은 모두 1, 4겹은 모두 2다
- 검증 겹이 전체 분포를 대표하지 못한다

StratifiedKFold는 모든 검증 겹에 세 클래스를 고르게 담는다

StratifiedKFold(n_splits=5) — 같은 데이터, 같은 반복문

[입력] 프롬프트 · Claude Code (Code 탭)

같은 iris 데이터로 StratifiedKFold(n_splits=5)를 시험해 줘.
1. 앞 실험과 같은 반복문으로 겹마다 훈련·검증 레이블을 출력해 줘.
2. 검증 겹별 클래스 개수를 표로 정리해 KFold 표와 나란히 보여 줘.



검토할 것

- 모든 검증 겹이 클래스별 10개씩인가?
- KFold 실험과 데이터·반복문이 같은가?

출력 읽기

- 모든 겹의 검증 레이블이 0·1·2를 같은 비율로 담는다
- 각 겹이 전체 데이터셋을 대표한다
- 분류 문제의 기본 분할 방식이 된다

교차검증 점수는 겹별 검증 정확도의 평균이다

랜덤 포레스트(n_estimators=100, max_depth=10)를 StratifiedKFold 5겹으로 검증한다

[입력] 프롬프트 · Claude Code (Code 탭)

```
iris 데이터로 랜덤 포레스트를 StratifiedKFold(n_splits=5)로 검증해 줘.
- 모델: n_estimators=100, max_depth=10, random_state=2019
- 겹마다 훈련·검증 정확도를 소수 둘째 자리 %로 출력해 줘.
- 마지막에 검증 정확도 평균을 Cross Validation Score로 출력해 줘.
```

```
0 Fold, train Accuracy : 100.00%, validation Accuracy : 96.67%
1 Fold, train Accuracy : 100.00%, validation Accuracy : 96.67%
2 Fold, train Accuracy : 100.00%, validation Accuracy : 93.33%
3 Fold, train Accuracy : 100.00%, validation Accuracy : 96.67%
4 Fold, train Accuracy : 100.00%, validation Accuracy : 100.00%
Cross Validation Score : 96.67%
```

검토할 것

- 겹마다 모델을 새로 학습했는가?
- 평균을 검증 정확도로 냈는가?
- 훈련 정확도 100%를 성능으로 읽지 않았는가?

결과

- 훈련 정확도는 모든 겹에서 100%다
- 검증 정확도는 93.33~100%로 겹마다 다르다
- 평균 96.67%가 교차검증 점수(Cross Validation Score)다

cross_val_score는 K겹 교차검증 점수를 한 번에 계산한다

cv에 숫자를 주면 그만큼 겹을 만들고, fold 객체를 주면 그 객체대로 분할한다

[입력] 프롬프트 · Claude Code (Code 탭)

```
정렬된 iris 데이터에서 cross_val_score로 세 가지를 비교해 줘.
- 모델: RandomForestClassifier(n_estimators=100,
max_depth=10, random_state=2019)
- cv=5, cv=KFold(n_splits=5), cv=StratifiedKFold(n_splits=5)
- 평균 정확도를 % 표로 보여 줘.
```

검토할 것

- cv=5와 StratifiedKFold의 값이 같은가?
- KFold가 가장 낮은가?
- 값은 모델 설정과 버전에 따라 달라진다. 표에 적었는가?

랜덤 포레스트 k-Fold CV Score(Acc) : 96.67%

랜덤 포레스트 k-Fold CV Score(Acc) : 91.33%

랜덤 포레스트 Stratify k-Fold CV Score(Acc) : 96.67%

- 겹 개수만큼 점수를 돌려주고, 그 평균으로 모델 성능을 평가한다. 분류 모델에는 기본값으로 StratifiedKFold를 적용한다
- 앞 쪽과 같은 랜덤 포레스트가 정렬된 데이터에서 KFold 객체로는 91.33%, StratifiedKFold로는 96.67%다. cv=5(기본 층화)도 96.67%로 앞 쪽 결과와 같다

분할 방식은 섞는지와 시간 순서를 지키는지로 갈린다

데이터 10개(0~9)를 기준으로 본 sklearn의 분할 방식

방식	검증 데이터를 만드는 법	분할 수
train_test_split	한 번 무작위로 나눈다(예: 훈련 80%)	1
KFold	순서대로 5겹으로 자르고 한 겹씩 검증한다	5
KFold(shuffle=True)	먼저 섞은 뒤 5겹으로 자른다	5
LeaveOneOut	한 개씩 빼서 검증한다	10
LeavePOut(p=2)	두 개씩 가능한 모든 조합을 뺀다	45
ShuffleSplit	섞어서 일정 개수를 뽑는 일을 여러 번 한다	3
TimeSeriesSplit	과거로 학습하고 바로 다음 시점으로 검증한다	5

K겹으로 나누면 모든 표본이 한 번씩 검증된다

train_test_split은 한 번만 나눠 운이 개입하고, KFold(n_splits=5)는 순서대로 2개씩 검증한다

train_test_split

```
data = list(range(1, 11))
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
train_test_split(data, train_size=.8)
[[8, 7, 4, 10, 1, 3, 5, 2], [6, 9]]
```

훈련 80%와 검증 20%를 한 번 무작위로 뽑는다. 어떤 표본이 검증에 들어가느냐에 따라 점수가 흔들린다

KFold

```
kf = KFold(n_splits=5)
for train, validate in kf.split(data):
    print(train, validate)
[2 3 4 5 6 7 8 9] [0 1]
[0 1 4 5 6 7 8 9] [2 3]
[0 1 2 3 6 7 8 9] [4 5]
[0 1 2 3 4 5 8 9] [6 7]
[0 1 2 3 4 5 6 7] [8 9]
```

0·1, 2·3, …, 8·9를 차례로 검증한다. 데이터가 정렬돼 있으면 검증 겹이 한쪽으로 치우친다

섞은 K겹은 정렬 문제를 피하고, LOO는 표본을 하나씩 뺀다

KFold(shuffle=True, random_state=42)와 LeaveOneOut

KFold with shuffle

```
kf = KFold(n_splits=5, shuffle=True, random_state=42)
for train, validate in kf.split(data):
    print(train, validate)
[0 2 3 4 5 6 7 9] [1 8]
[1 2 3 4 6 7 8 9] [0 5]
[0 1 3 4 5 6 8 9] [2 7]
[0 1 2 3 5 6 7 8] [4 9]
[0 1 2 4 5 7 8 9] [3 6]
```

섞은 뒤 자르므로 검증 겹이 1·8, 0·5처럼 흩어진다.
재현을 위해 random_state를 고정한다

Leave-one-out CV

```
loo = LeaveOneOut()
for train, validate in loo.split(data):
    print(train, validate)
[1 2 3 4 5 6 7 8 9] [0]
[0 2 3 4 5 6 7 8 9] [1]
...
[0 1 2 3 4 5 6 7 9] [8]
[0 1 2 3 4 5 6 7 8] [9]
```

표본 수만큼(10번) 학습한다. 데이터가 작을 때 쓰지만
표본이 많으면 계산이 무겁다

LPO는 모든 조합을, ShuffleSplit은 무작위로 뽑는다

LeavePOut(p=2)와 ShuffleSplit(n_splits=3, test_size=2)

Leave-p-out CV

```
lpo = LeavePOut(p=2)
for train, validate in lpo.split(data):
    print(train, validate)
[2 3 4 5 6 7 8 9] [0 1]
[1 3 4 5 6 7 8 9] [0 2]
...
[0 1 2 3 4 5 6 8] [7 9]
[0 1 2 3 4 5 6 7] [8 9]
```

10개 중 2개를 빼는 모든 경우(45가지)를 검증한다. p가 커지면 조합 수가 폭발한다

Shuffle Split

```
ss = ShuffleSplit(n_splits=3, test_size=2, random_state=42)
for train, validate in ss.split(data):
    print(train, validate)
[5 0 7 2 9 4 3 6] [8 1]
[8 5 3 4 7 9 6 2] [0 1]
[0 6 8 5 3 7 1 4] [9 2]
```

매번 섞어서 2개를 검증으로 뽑는다. 같은 표본이 여러 번 뽑히거나 한 번도 안 뽑힐 수 있다

시계열에서는 과거로 학습하고 미래로 검증해야 한다

TimeSeriesSplit(n_splits=5) — 훈련 구간이 점점 늘고, 검증은 늘 그 바로 다음 시점이다

왜 섞으면 안 되나

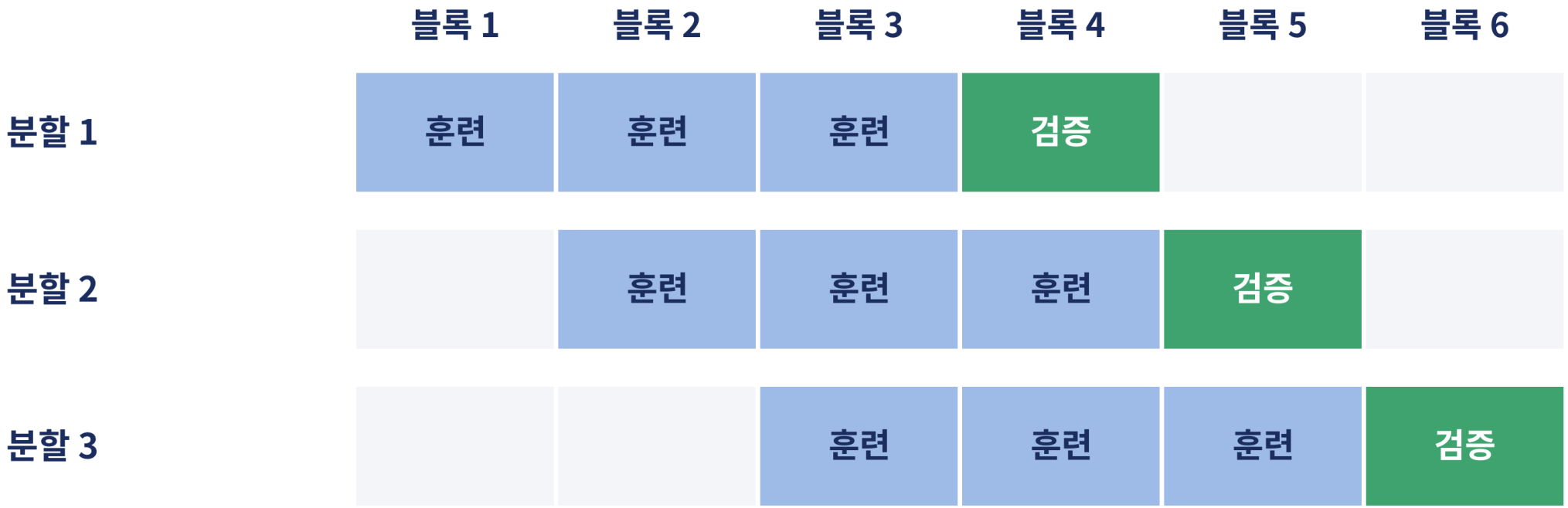
- 섞으면 미래 데이터로 학습하고 과거를 맞히는 셈이 된다
- 실제 운용에서는 미래를 볼 수 없으므로 검증 점수가 부풀려진다
- 그래서 0~4로 학습해 5를, 0~5로 학습해 6을 검증한다

```
tscv = TimeSeriesSplit(n_splits=5)
```

```
for train, validate in tscv.split(data):
    print(train, validate)
[0 1 2 3 4] [5]
[0 1 2 3 4 5] [6]
[0 1 2 3 4 5 6] [7]
[0 1 2 3 4 5 6 7] [8]
[0 1 2 3 4 5 6 7 8] [9]
```

워크포워드는 일정한 훈련 창을 한 칸씩 앞으로 민다

Walk Forward CV — 훈련셋을 k개 블록으로 나누고, 1~p 블록으로 학습해 p+1 블록을 검증하는 일을 반복한다



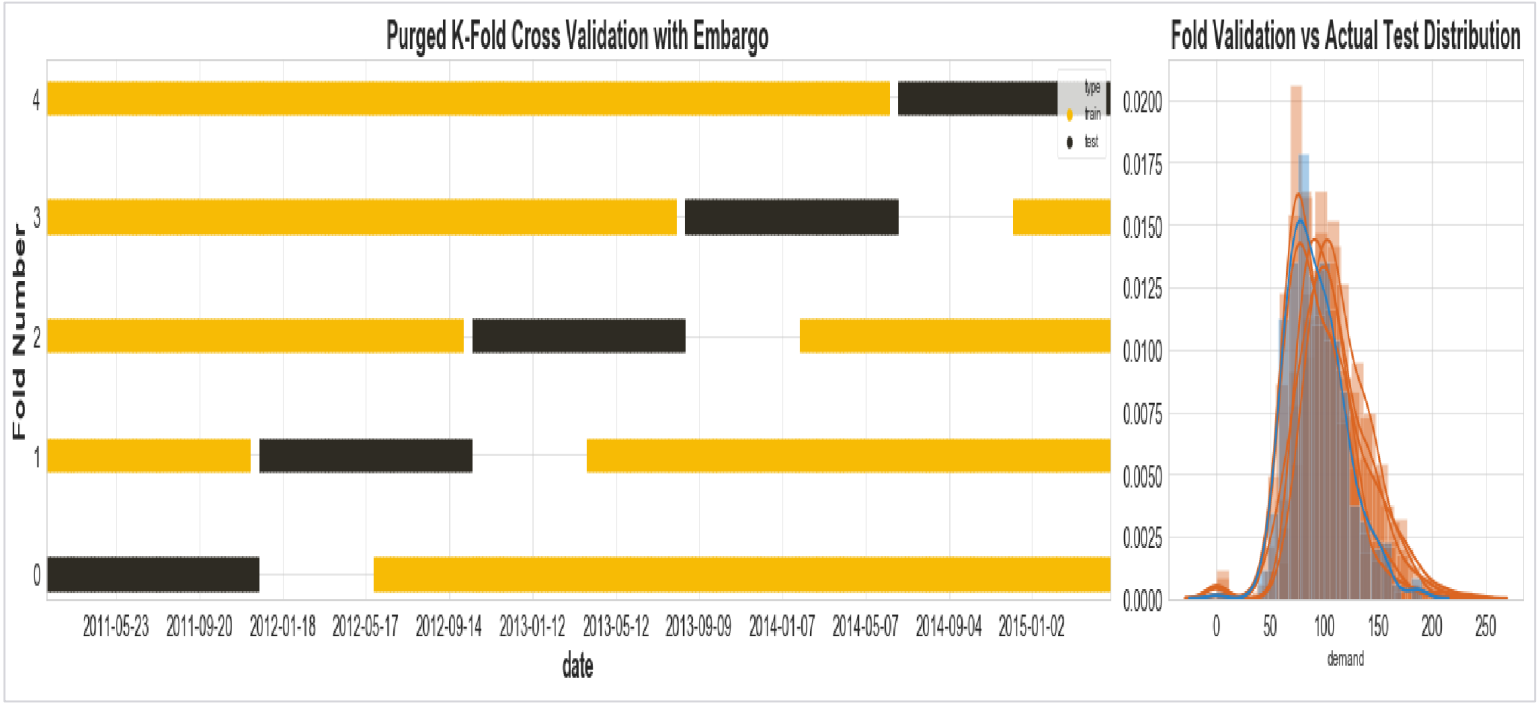
k = 6개 블록, 훈련 창 p = 3블록 → 창을 한 칸씩 밀어 k − p = 3개의 분할을 만든다. 검증은 늘 훈련보다 뒤다

퍼지와 엠바고를 넣은 K겹은 시계열에서도 모든 구간을 검증한다

Purged K-Fold CV with Embargo — 노랑은 훈련, 검정은 검증 겹, 오른쪽은 분포 비교다

그림 읽기

- 검증 겹(검정) 바로 뒤의 짧은 빈칸이 엠바고다
- 워크포워드와 달리 검증 겹 뒤의 데이터도 훈련에 쓴다
- 겹별 검증 분포가 실제 테스트 분포와 비슷한지 함께 본다

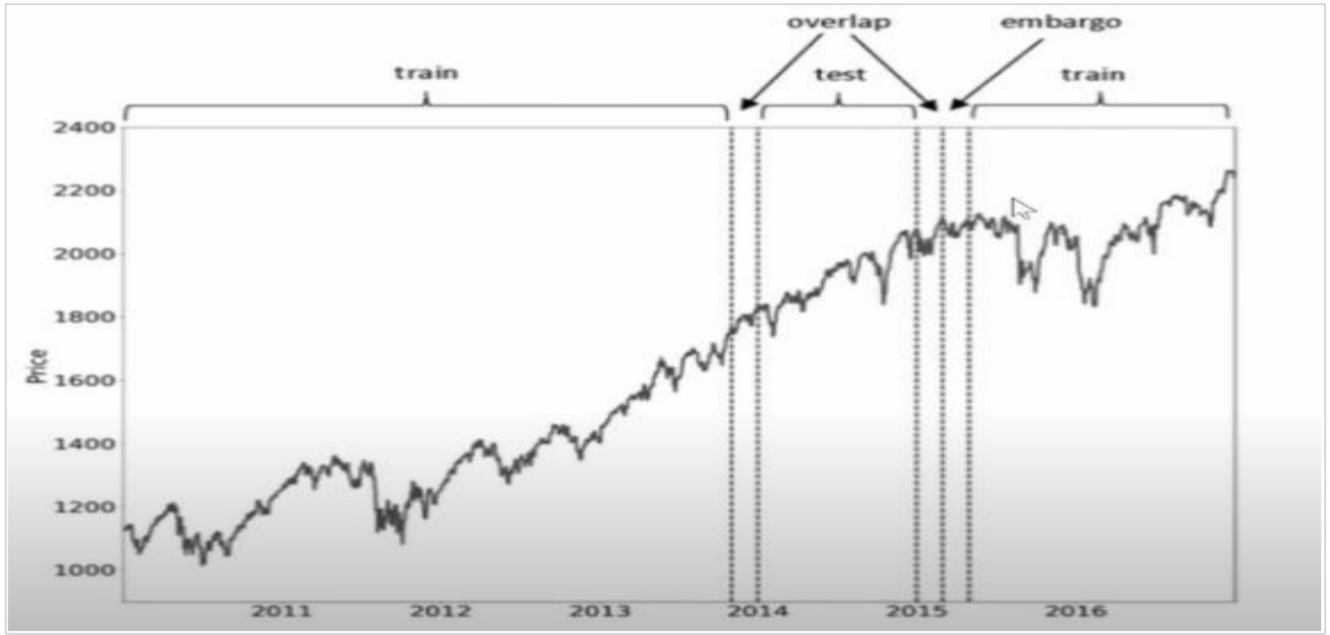


금융 시계열의 교차검증에는 세 가지 함정이 있다

테스트 구간 앞뒤의 겹침(overlap)과 엠바고 — 『Advances in Financial Machine Learning』 7장

문제점과 처방

- 레이블 구간이 테스트 구간과 겹치는 훈련 표본이 테스트 정보를 담는다 → 퍼지(purge)
- 테스트 직후 표본은 자기상관 때문에 테스트 정보를 담는다. 이 구간을 추가로 비운다 → 엠바고(embargo)
- 경로가 하나뿐이라 백테스트 결과가 한 번의 운에 좌우되고, 훈련셋도 작다 → 조합 경로 생성



퍼지는 겹친 레이블을, 엠바고는 테스트 직후를 훈련에서 뺀다

삼중 배리어처럼 레이블이 여러 날에 걸치면, 테스트와 겹친 훈련 표본이 답을 미리 본다

테스트 구간 앞뒤를 비워 미래 정보가 훈련으로 새지 않게 한다



퍼지(purge): 테스트 구간과 레이블 기간이 겹치는 훈련 표본을 지운다

엠바고(embargo): 테스트 직후 일정 기간의 표본을 훈련에서 뺀다 — 자기상관으로 정보가 새는 것을 막는다

조합 경로 교차검증은 백테스트 경로를 여러 개 만든다

Combinatorial Purged CV — N개 그룹 중 k개를 테스트로 고르는 모든 조합을 쓴다 (AFML 12장)

$$\underbrace{\varphi[N,k]}_{\text{경로 수}} = \frac{k}{N}\underbrace{\binom{N}{k}}_{\text{훈련·테스트 분할 수}}$$

N : 그룹 수, k : 테스트 그룹 수

$N=6,\ k=2$: $\binom{6}{2}=15$ 분할, $\varphi[6,2]=\frac{2}{6}\times 15=5$ 경로

	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15	Paths
G1	x	x	x	x	x											5
G2	x					x	x	x	x							5
G3		x				x				x	x	x				5
G4			x				x			x			x	x		5
G5				x				x			x		x		x	5
G6					x				x			x		x	x	5

FIGURE 12.1 Paths generated for $\varphi[6,2]=5$

표 읽기

- 행이 6개 그룹, 열이 15개 분할이다
- 각 그룹이 다섯 번 테스트되어 경로 다섯 개가 된다
- 성과를 하나가 아닌 분포로 본다

02

PART 2 · 파라미터 튜닝

파라미터 튜닝

그리드 서치 · 랜덤 서치 · 베이지안 하이퍼파라미터 최적화

하이퍼파라미터를 어떻게 정하느냐가 학습 성과를 좌우한다

그리드 서치(grid search)는 정해 둔 후보 값의 모든 조합을 교차검증으로 비교한다

그리드 서치

- 예: SVC는 소프트·하드 마진의 정도를 정하는 C, 커널 함수를 정하는 kernel, 특정 커널에서 얼마나 세세하게 볼지를 정하는 gamma에 따라 학습 성과가 달라진다
- 여기서는 GridSearchCV로 랜덤 포레스트의 n_estimators와 max_depth 중 가장 좋은 값을 찾는다
- GridSearchCV는 sklearn의 model_selection 패키지에 있다

참고 — scikit-optimize (skopt)

- sklearn에는 없지만 scikit-optimize(skopt) 라이브러리가 있다
- 각 파라미터 값의 최대·최소 범위와 값의 분포 스케일을 정해 주면 튜닝을 자동화한다
- skopt의 BayesSearchCV로 베이지안 하이퍼파라미터 최적화를 할 수 있다

그리드 서치는 20개 조합을 모두 비교해 최적 조합을 찾는다

n_estimators 5개 × max_depth 4개 = 20개 조합 — 최적은 max_depth 5, n_estimators 50이다

[입력] 프롬프트 · Claude Code (Code 탭)

```
iris 데이터로 랜덤 포레스트를 GridSearchCV로 튜닝해 줘.
- n_estimators: 50, 150, 200, 300, 400
- max_depth: 2, 5, 10, 20
- random_state=2019, cv=StratifiedKFold(n_splits=5)
- best_score_(%)와 best_params_를 출력해 줘.
- 20개 조합의 점수를 표로 저장해 줘.
```

```
GridSearchCV best score : 96.67%,

best_params : {'max_depth': 5, 'n_estimators': 50}
```

검토할 것

- 20개 조합을 모두 평가했는가?
- 최고 점수가 동점이면 어떤 조합이 뽑혔는가?
- 최적값이 격자의 끝에 걸리지 않았는가?

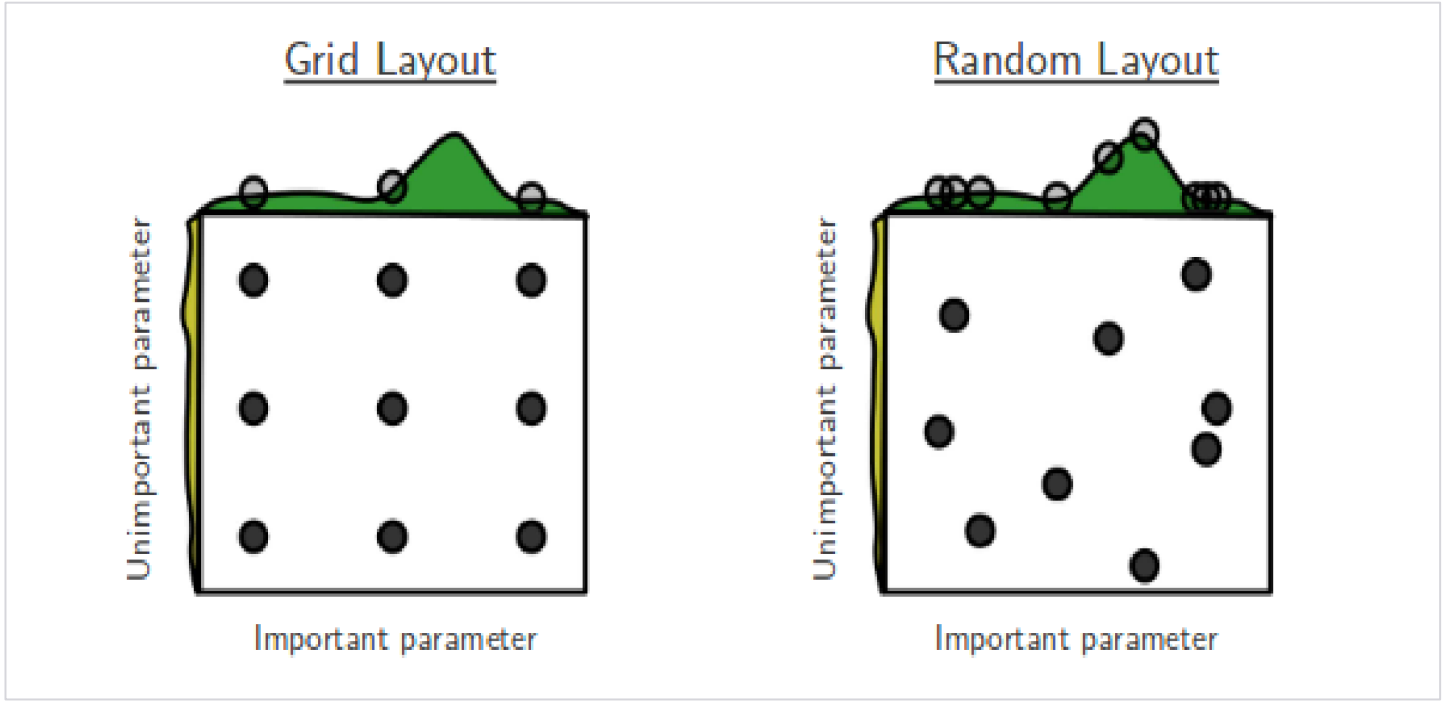
best score 96.67% — 앞의 교차검증(100개·깊이 10)과 같은 점수를 더 작은 모델로 얻었다

같은 횟수를 시도해도 랜덤 서치가 중요한 값을 더 다양하게 본다

Grid Layout vs Random Layout — 9번 시도, 위쪽 곡선은 중요한 파라미터에 대한 성능

그리드 서치 대 랜덤 서치

- 그리드는 9번을 시도해도 중요한 파라미터 값은 3개밖에 보지 못한다
- 랜덤은 9번 모두 다른 값을 보므로 성능 곡선의 봉우리에 더 가까이 간다
- sklearn.model_selection의 RandomizedSearchCV를 쓰며, 사용법은 GridSearchCV와 같다

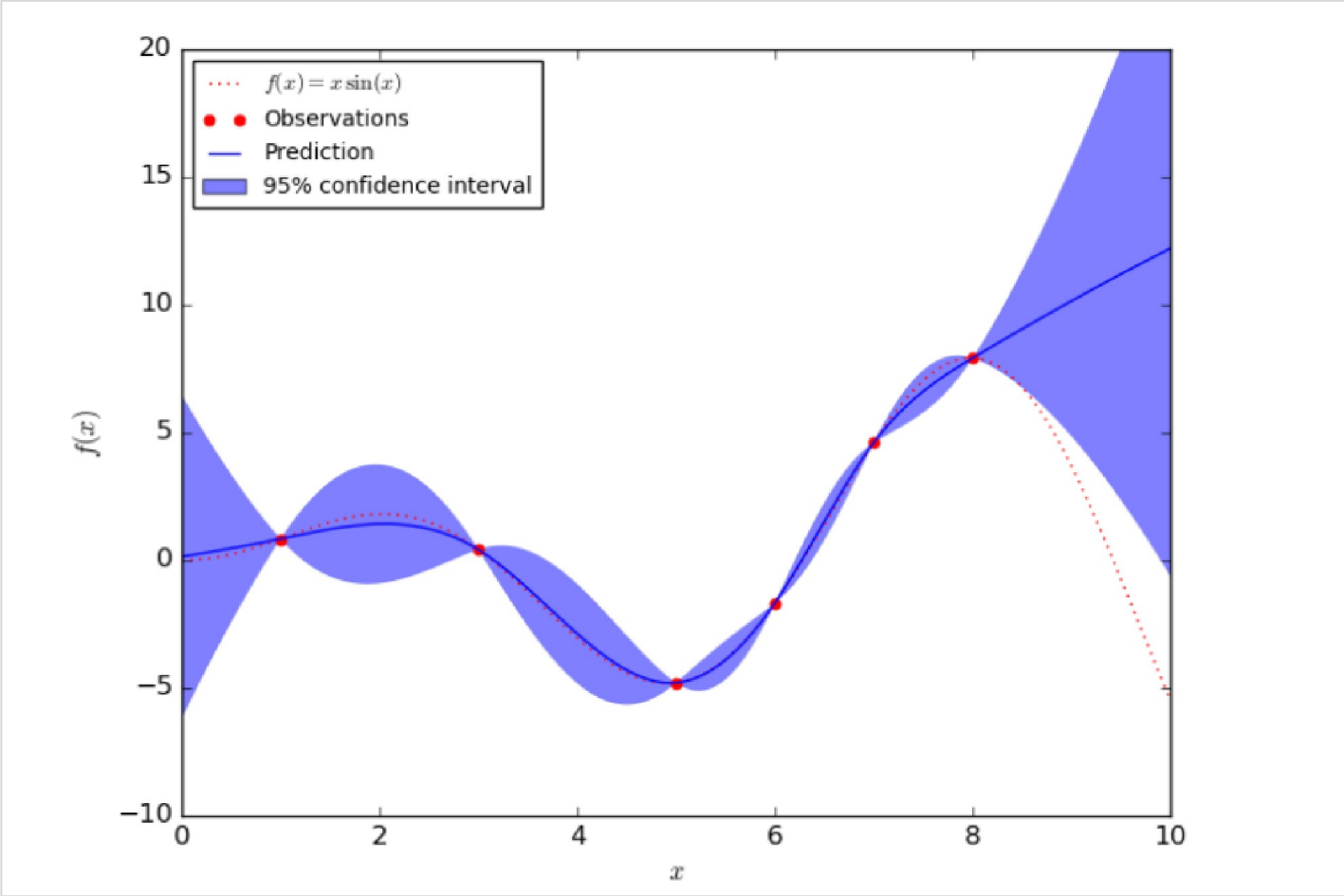


가우시안 프로세스는 목적함수를 평균과 불확실성으로 추정한다

대리모델(surrogate model) — 빨간 점선 = 실제 함수, 빨간 점 = 관측, 파란 띠 = 95% 신뢰구간

그림 읽기

- 하이퍼파라미터 성능 (목적함수)은 계산 비용이 커서 몇 점만 관측한다
- 관측점 근처에서는 신뢰구간이 좁고, 관측점에서 멀수록 넓어진다
- 이 대리모델로 다음에 관측할 점을 고른다

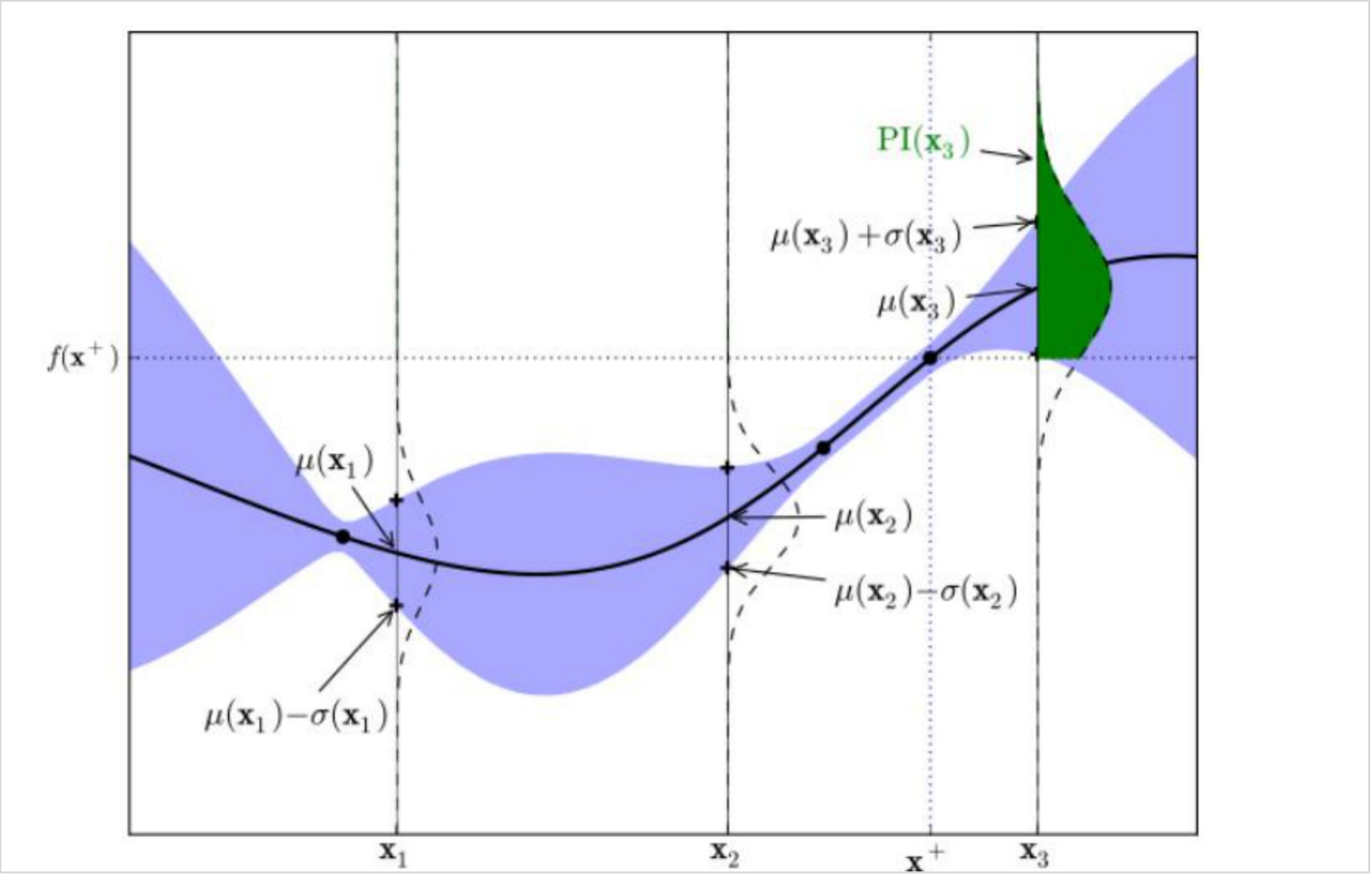


가우시안 프로세스는 각 점의 예측을 정규분포로 나타낸다

가우시안 프로세스 설명 — 점선 높이는 지금까지의 최고값, 초록 면적(PI)은 그 값을 넘을 확률

그림 읽기

- 세 후보 점마다 평균과 ±1 표준편차가 표시돼 있다
- 오른쪽 끝 점은 평균이 현재 최고값보다 높고 불확실성도 크다
- 초록 면적이 클수록 그 점을 관측할 가치가 크다



획득함수는 기대개선이 가장 큰 후보를 다음 관측점으로 고른다

획득함수(acquisition function) = 기대개선(EI, Expected Improvement)을 최대로 하는 함수

$$x^* = \arg\max_{x_i^{\text{new}} \in X^{\text{new}}} \underbrace{\text{EI}(x_i^{\text{new}})}_{\text{기대개선}},$$

$$\text{EI}(x_i^{\text{new}}) = \mathbb{E}\Big[\max\big(0,\ \underbrace{f(x_i^{\text{new}})}_{\text{새 후보의 성능}} - \underbrace{\max_{x_j \in X} f(x_j)}_{\text{지금까지 최고}}\big)\Big]$$

읽는 법

- 새 후보가 지금까지의 최고 성능을 넘는 만큼만 개선으로 센다. 넘지 못하면 개선은 0이다
- 그 개선량의 기댓값이 가장 큰 후보를 다음에 관측한다 (출처: 고려대 DMQA 세미나)

기대개선은 활용 항과 탐험 항을 더해 계산한다

가우시안 프로세스의 평균과 표준편차로 기대개선을 닫힌 형태로 계산한다

$$\mathrm{EI}(x) = \begin{cases} \underbrace{\big(\mu(x) - f(x^+) - \xi\big)\Phi(Z)}_{\text{활용 } (exploitation)} + \underbrace{\sigma(x)\,\phi(Z)}_{\text{탐험 } (exploration)} & \sigma(x) > 0 \\ 0 & \sigma(x) = 0 \end{cases} \qquad Z = \frac{\mu(x) - f(x^+) - \xi}{\sigma(x)}$$

$\mu(x)$: 대리모델이 예측한 평균 $\sigma(x)$: 예측의 표준편차 $f(x^+)$: 지금까지 관측한 최고값
ξ : 탐험 여유(작은 양수) Φ : 표준정규 누적분포 ϕ : 표준정규 밀도

활용 (Exploitation)

- 평균이 현재 최고값보다 높은 곳을 고른다

탐험 (Exploration)

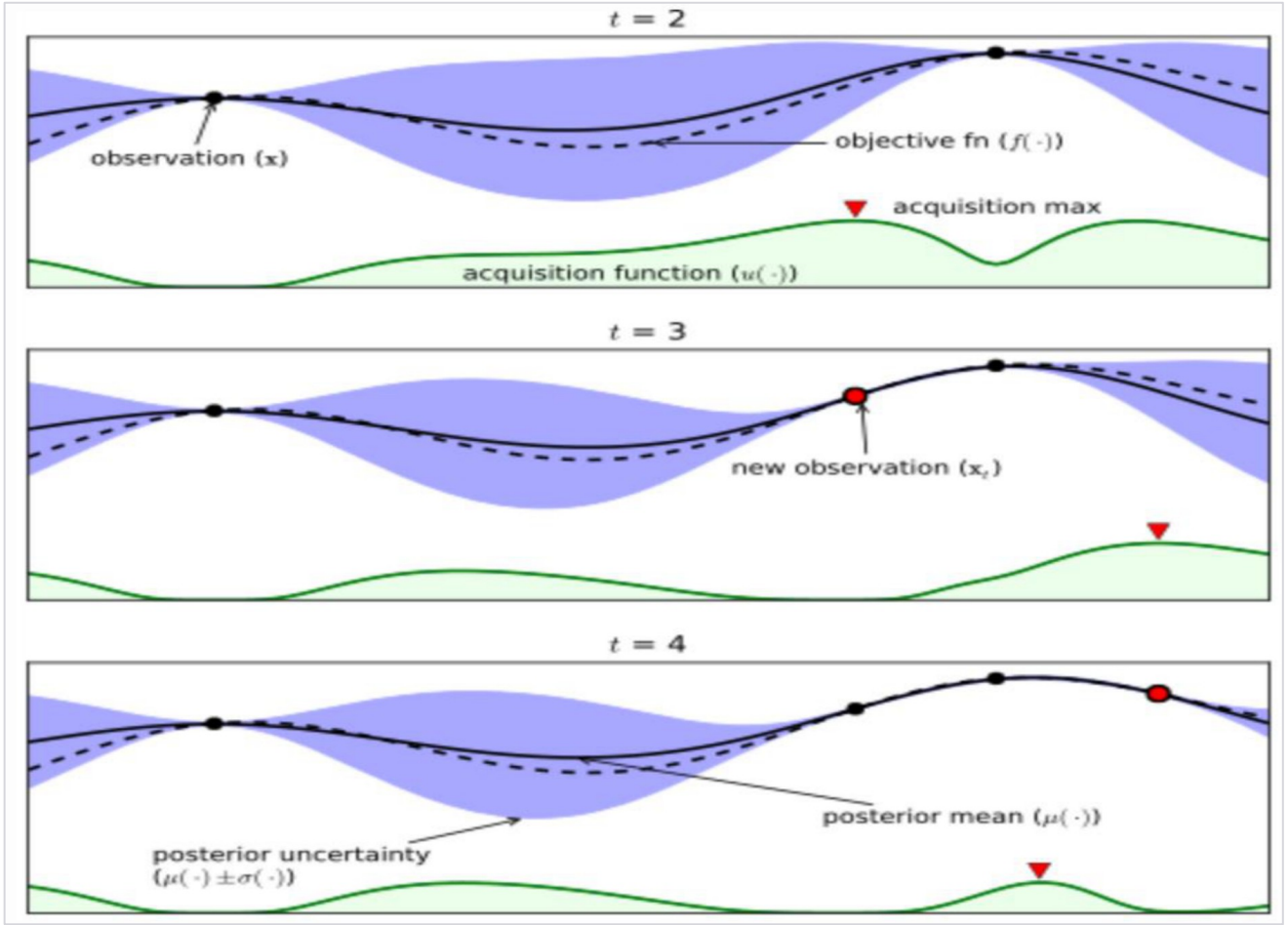
- 불확실성이 큰, 아직 덜 본 곳을 고른다

관측이 쌓일수록 획득함수가 고르는 점이 최적점에 다가간다

t = 2, 3, 4 — 획득함수 최대점(빨간 삼각형)을 관측하고 대리모델을 다시 추정한다

범례

- 검은 점선: 실제 목적함수
- 검은 실선: 추정한 함수의 평균
- 보라 영역: 추정한 함수의 표준편차
- 초록 영역: 획득함수



베이지안 최적화는 교차검증 점수를 목적함수로 삼는다

LightGBM 예 — lgb.cv의 AUC 평균을 최대화하도록 BayesianOptimization이 6개 파라미터를 고른다

[입력] 프롬프트 · Claude Code (Code 탭)

```
LightGBM을 베이지안 최적화로 튜닝해 줘(이진 분류, X, y).
- 목적함수: lgb.cv의 AUC 평균 최대화
  (nfold=3, 층화, seed=6, n_estimators=100, learning_rate=0.05,
early_stopping_round=100)
- 범위: num_leaves 24~45, feature_fraction 0.1~0.9, bagging_fraction
0.8~1, max_depth 5~8.99, min_split_gain 0.001~0.1, min_child_weight
5~50
- init_points=5, n_iter=10, random_state=0
- 최적 파라미터와 AUC를 출력해 줘.
```

검토할 것

- num_leaves·max_depth를 정수로 반올림했는가?
- feature_fraction 등을 0~1 안으로 제한했는가?
- lgb.cv 결과의 AUC 열 이름이 설치 버전과 맞는가?
- 최적값이 범위 끝에 붙지 않았는가?

init_points로 무작위 탐색을 먼저 하고, n_iter만큼 획득함수가 고른 점을 차례로 관측한다

03

PART 3 · 앙상블

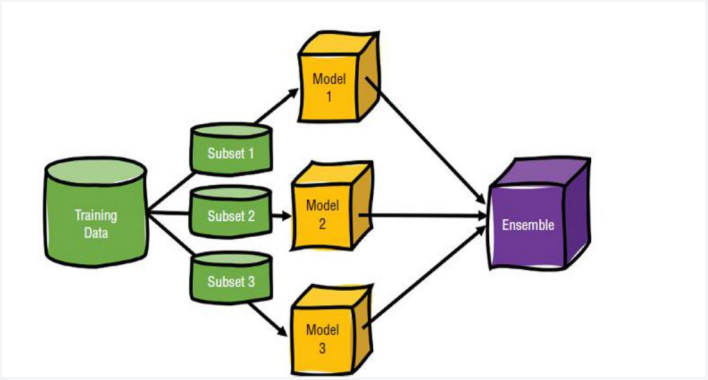
앙상블

배깅 · 부스팅 · 스태킹, 투표 앙상블과 평균 블렌딩

앙상블은 모델을 묶는 방식에 따라 세 갈래로 나뉜다

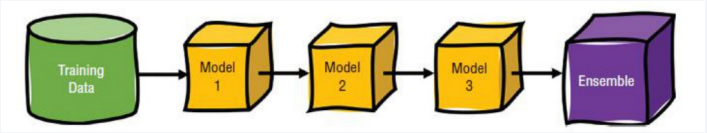
단일 모델의 성과도 중요하지만, 서로 다른 모델의 다양성을 살리는 앙상블은 머신러닝의 꽃이다

배깅 (Bagging)



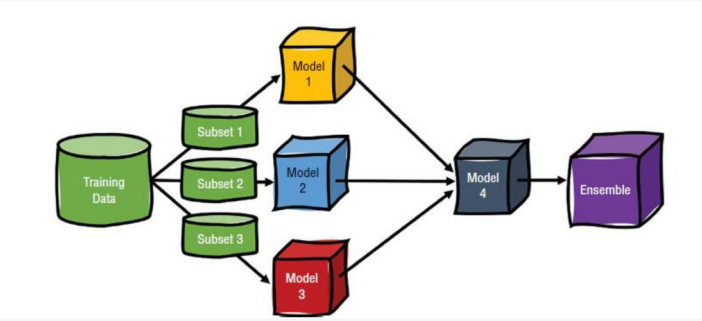
독립적으로 훈련한 동질의 모델을 결합한다

부스팅 (Boosting)



동질의 모델을 순차적으로 적용한다

스태킹 (Stacking)



메타 모델로 독립적으로 훈련한 모델을 결합한다

실습에서는 가장 많이 쓰는 앙상블 세 가지를 다룬다

Voting Ensemble · Average Blending · Stacking

1

투표 앙상블 (Voting Ensemble)

각 모델이 투표해 다수결로 클래스를 고른다. sklearn이 지원해 쓰기 쉽다

2

평균 블렌딩 (Average Blending)

모델들의 예측값 또는 클래스 확률을 평균해 합친다. 캐글에서 가장 많이 쓴다

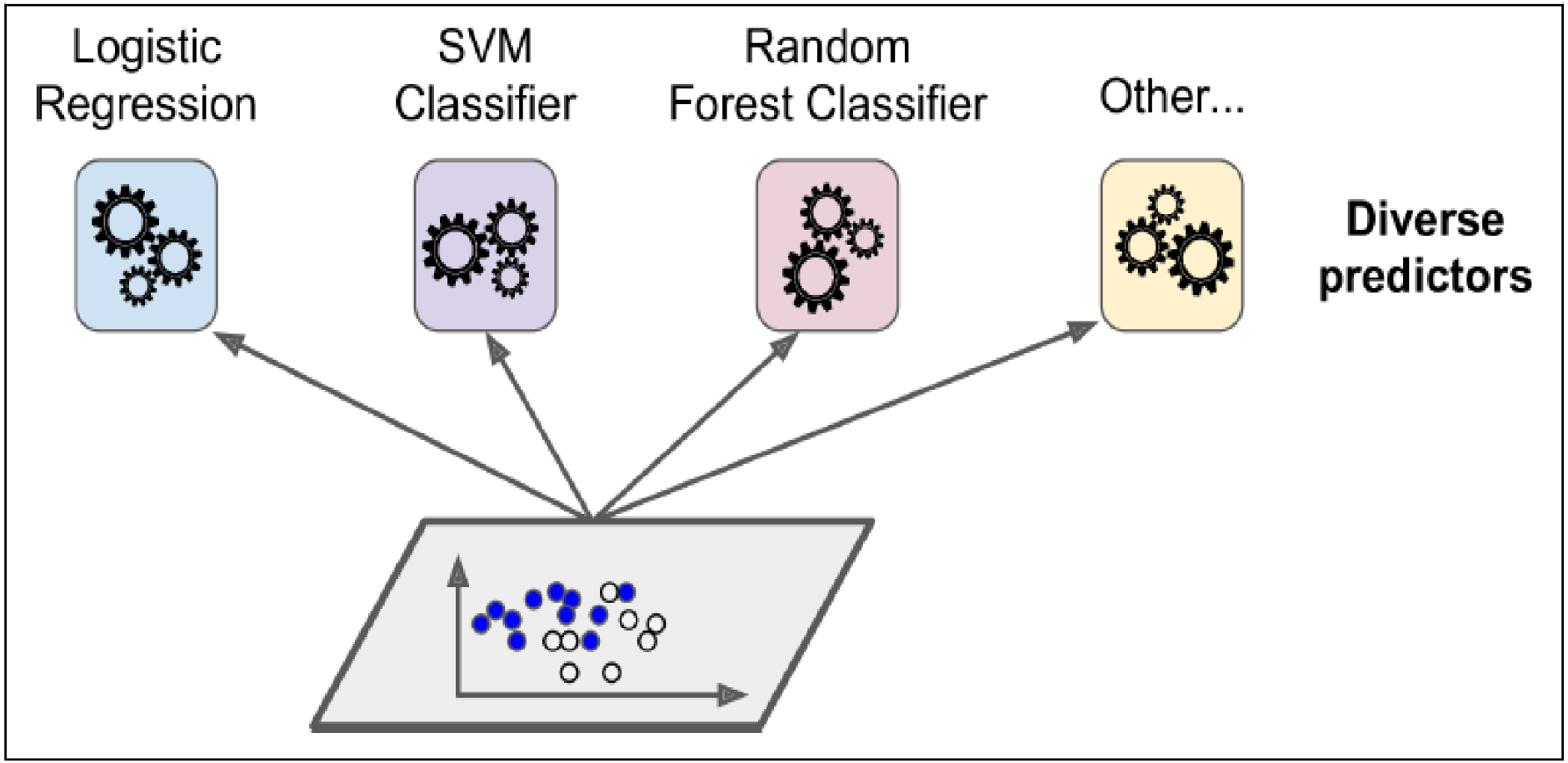
3

스태킹 (Stacking)

예측기들의 예측을 입력으로 받는 메타 학습기(블렌더)를 훈련한다

투표 기반 분류기는 같은 데이터로 서로 다른 예측기를 훈련한다

여러 분류기 훈련하기 — 로지스틱 회귀, SVM, 랜덤 포레스트 등 다양한 예측기(diverse predictors)

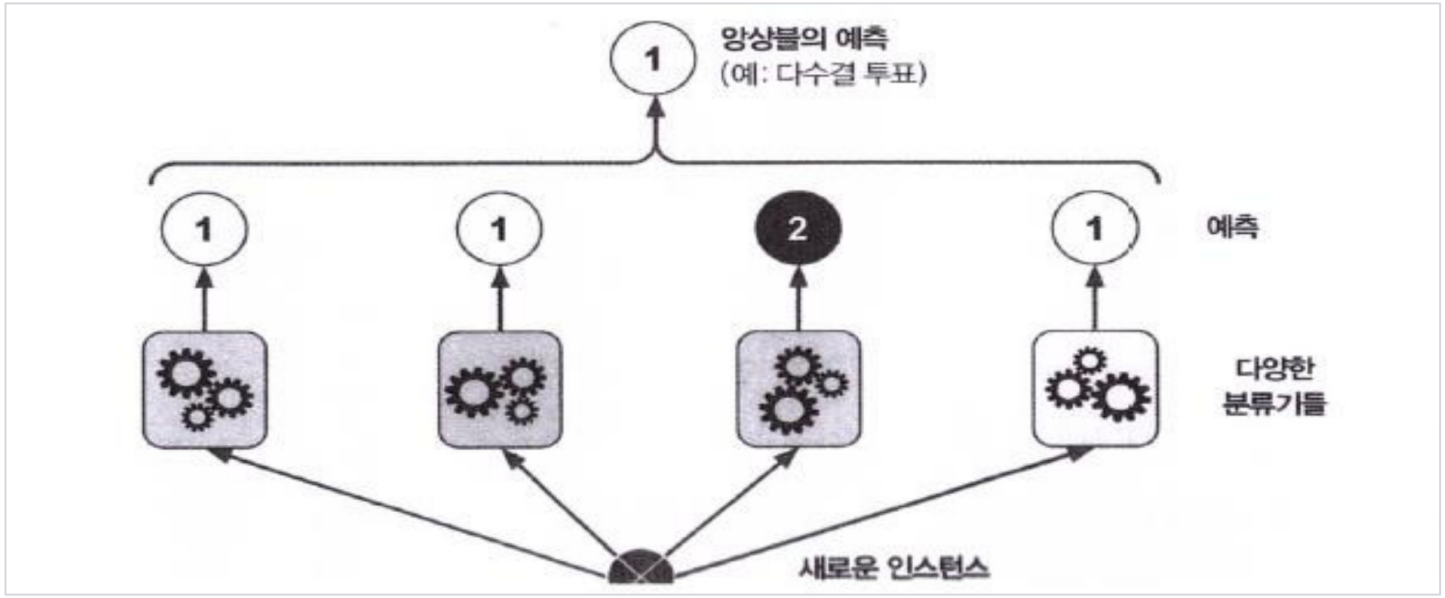


약한 학습기라도 충분히 많고 다양하면 강한 학습기가 된다

직접 투표(hard voting) 분류기의 예측 — 새로운 인스턴스에 대해 다수결로 클래스를 고른다

투표 앙상블

- 각 모델이 투표하고 다수결로 클래스를 고른다 (그림: 1·1·2·1 → 1)
- VotingClassifier는 sklearn의 ensemble 패키지에 있다
- 실습은 Adult 데이터셋으로 한다



성능이 낮은 모델이 섞이면 앙상블이 단일 모델보다 못하다

Voting 앙상블(voting='soft') — 로지스틱 회귀 + 랜덤 포레스트, Adult 데이터셋

[입력] 프롬프트 · Claude Code (Code 탭)

```
Adult 데이터셋으로 투표 앙상블을 시험해 줘.
- voting='soft'
- 로지스틱 회귀(random_state=2019) + 랜덤 포레스트
(n_estimators=100, max_depth=10, random_state=2019)
- 비교: 같은 설정의 단일 랜덤 포레스트, 로지스틱 회귀 단독
- 테스트 20%(random_state=2019)로 나눠 세 정확도를 출력해 줘.
```

검토할 것

- 세 모델이 같은 테스트셋을 썼는가?
- 로지스틱 회귀 단독 정확도는 얼마인가?
- 앙상블 안과 단일 랜덤 포레스트의 설정이 같은가?

```
Voting Ensemble Acc : 82.51%

Single Random Forest Acc : 83.56%
```

결과

- Voting 앙상블 82.51% (RF 100개·깊이 10)
- 단일 랜덤 포레스트 83.56% (RF 50개·깊이 5, 설정이 다름)
- 같은 설정 재계산(테스트 20%, 시드 2019): 앙상블 82.21%, 단일 RF 85.48%, 로지스틱 회귀 78.30%. 이 설정에서는 앙상블이 단일 RF보다 낮다

평균 블렌딩은 n개 모델의 예측을 평균해 하나로 합친다

Average 블렌딩 — 캐글에서 가장 많이 쓰는 기법

회귀

- 모델들이 예측한 값을 n으로 나눠 합친다

분류

- 각 클래스의 확률을 n으로 나눠 합친다
- 그중 가장 높은 확률값을 갖는 클래스를 고른다

평균 블렌딩은 겹마다 학습한 모델의 예측 확률을 평균한다

Average 블렌딩 앙상블 — StratifiedKFold 5겹의 랜덤 포레스트 예측 확률을 skf.n_splits로 나눠 더한다

[입력] 프롬프트 · Claude Code (Code 탭)

```
Adult 데이터셋으로 평균 블렌딩을 구현해 줘.
- 모델: 랜덤 포레스트(n_estimators=50, max_depth=5,
random_state=2019)
- x_train을 StratifiedKFold(n_splits=5)로 나눠 겹마다 학습해 줘.
- 겹마다 훈련·검증 정확도를, 끝에 평균을 Cross Validation Score로
출력해 줘.
- x_test 예측 확률을 skf.n_splits로 나눠 더해 y_pred를 만들어 줘.
- 같은 x_test에서 단일 모델과 테스트 정확도를 비교해 줘.
```

검토할 것

- x_test가 학습 겹에 섞이지 않았는가?
- 예측 확률을 skf.n_splits로 나눠 더했는가?
- 테스트 점수끼리 비교했는가? CV 점수와 섞지 않았는가?

```
0 Fold, train Accuracy : 83.37%, validation Accuracy : 83.79%
1 Fold, train Accuracy : 84.00%, validation Accuracy : 83.24%
2 Fold, train Accuracy : 83.95%, validation Accuracy : 83.70%
3 Fold, train Accuracy : 83.87%, validation Accuracy : 83.79%
4 Fold, train Accuracy : 83.96%, validation Accuracy : 83.58%
Cross Validation Score : 83.62%

Single Random Forest Acc : 83.56%
```

결과

- 겹별 검증 정확도 83.24~83.79%
- 교차검증 점수 83.62%
- 83.62%는 CV 점수, 83.56%는 테스트 점수라 직접 견주지 않는다. 같은 테스트셋 재계산: 블렌딩 84.51%, 단일 RF 84.56%로 비슷하다

04

PART 4 · 스태킹

스태킹

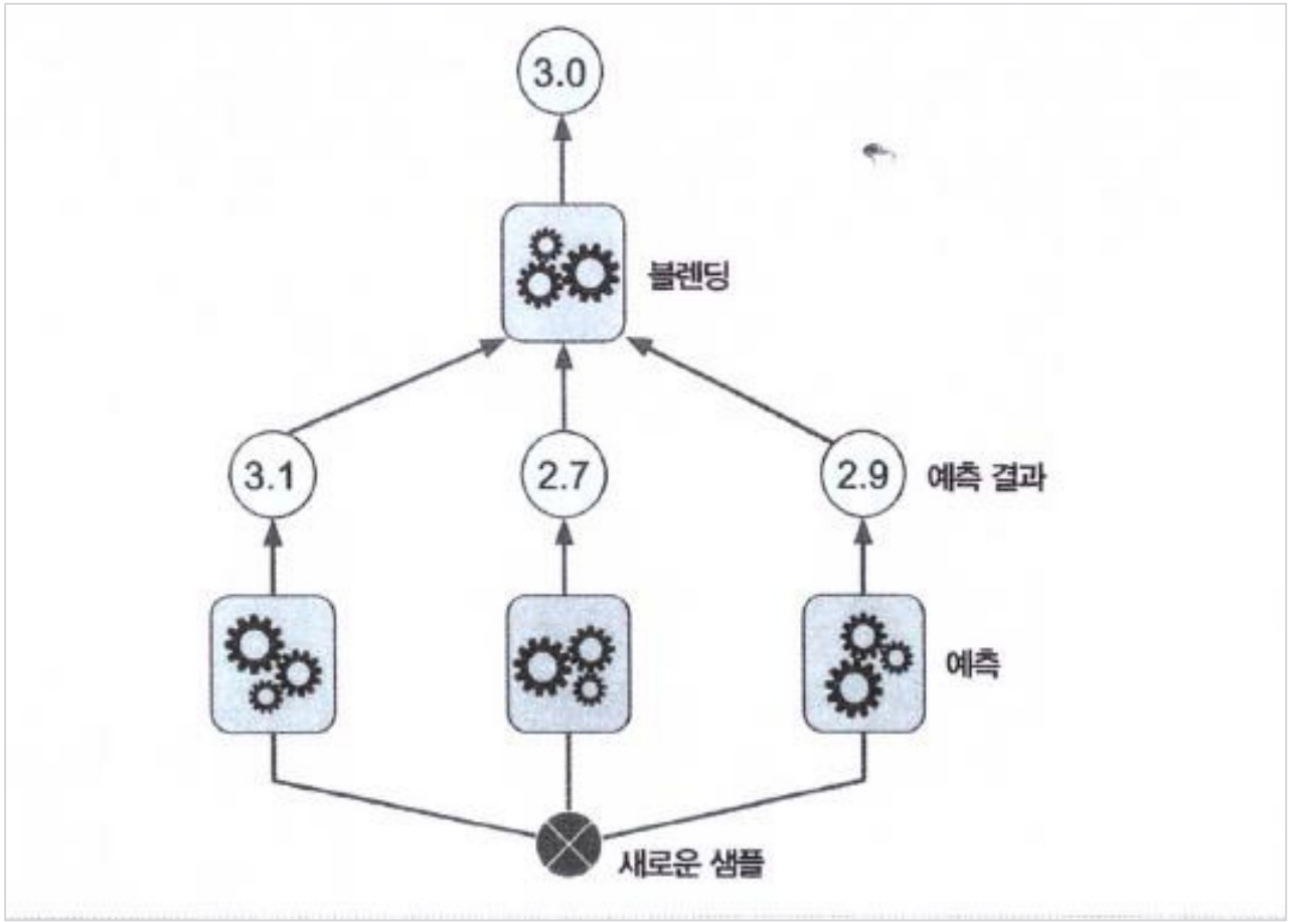
예측기의 예측을 취합하는 메타 모델과 CV 기반 스태킹

스태킹은 예측기들의 예측을 합치는 모델을 따로 훈련한다

Stacked Generalization의 줄임말 — 마지막 예측기를 블렌더(blender) 또는 메타 학습기라 부른다

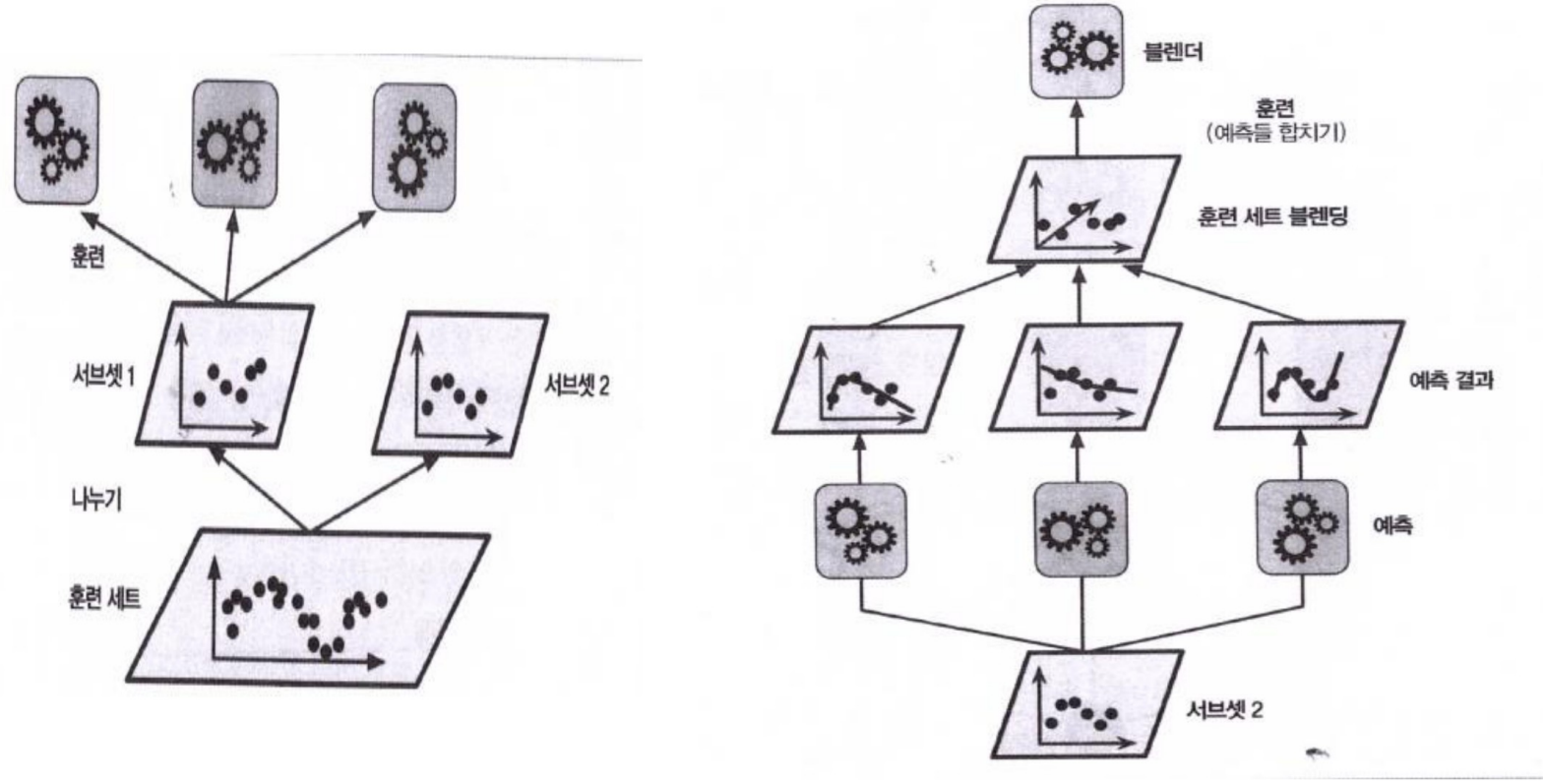
그림 읽기

- 세 예측기가 새 샘플에 대해 각각 3.1, 2.7, 2.9를 예측한다
- 블렌더가 이 세 예측을 입력으로 받아 최종 예측 3.0을 낸다
- 투표·평균처럼 정해진 규칙으로 합치지 않고, 합치는 방법 자체를 학습한다



블렌더는 첫 번째 층이 보지 못한 홀드아웃셋으로 훈련한다

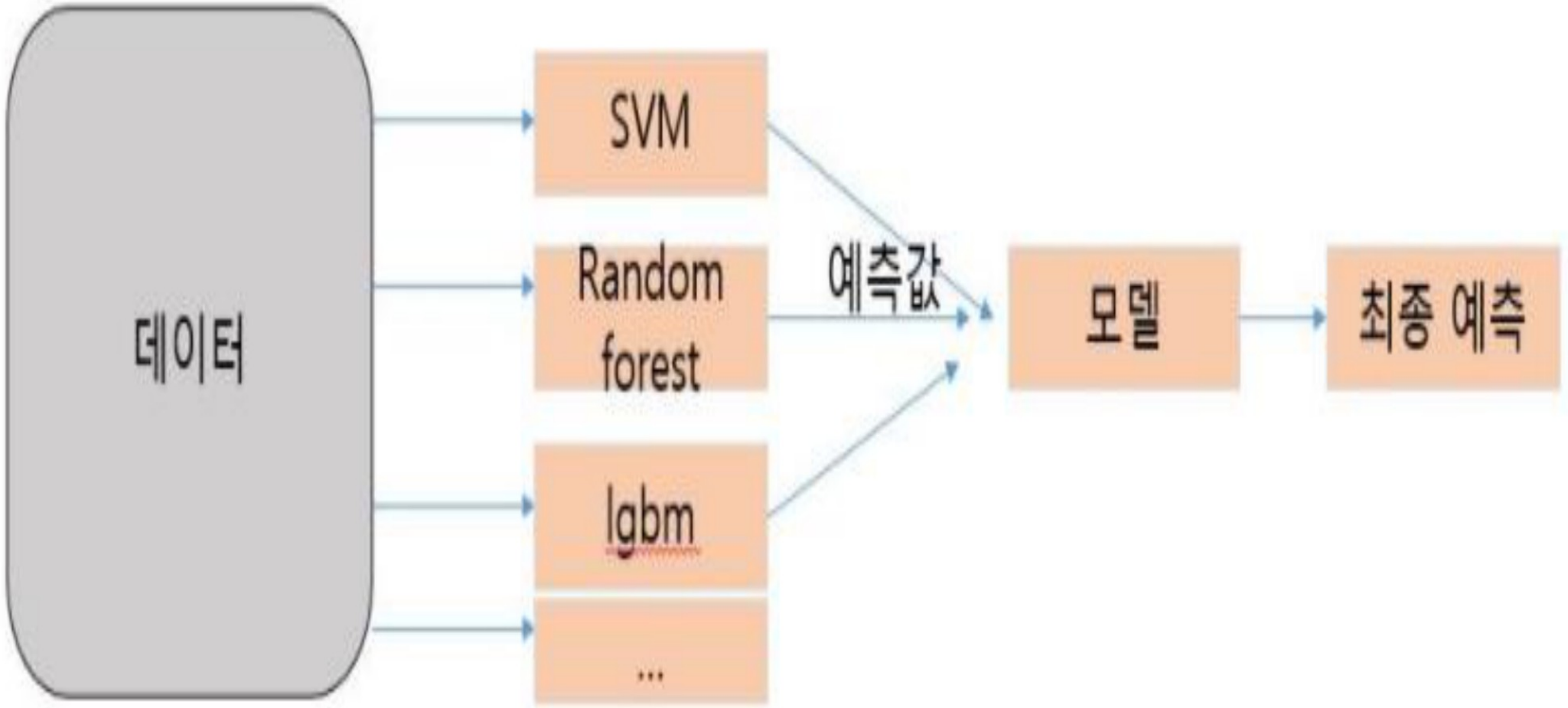
스태킹 훈련 과정 — 왼쪽: 첫 번째 레이어 훈련, 오른쪽: 블렌더 훈련



서브셋 1(훈련셋)로 첫 층을 훈련하고, 서브셋 2(홀드아웃셋) 예측으로 블렌더를 훈련한다

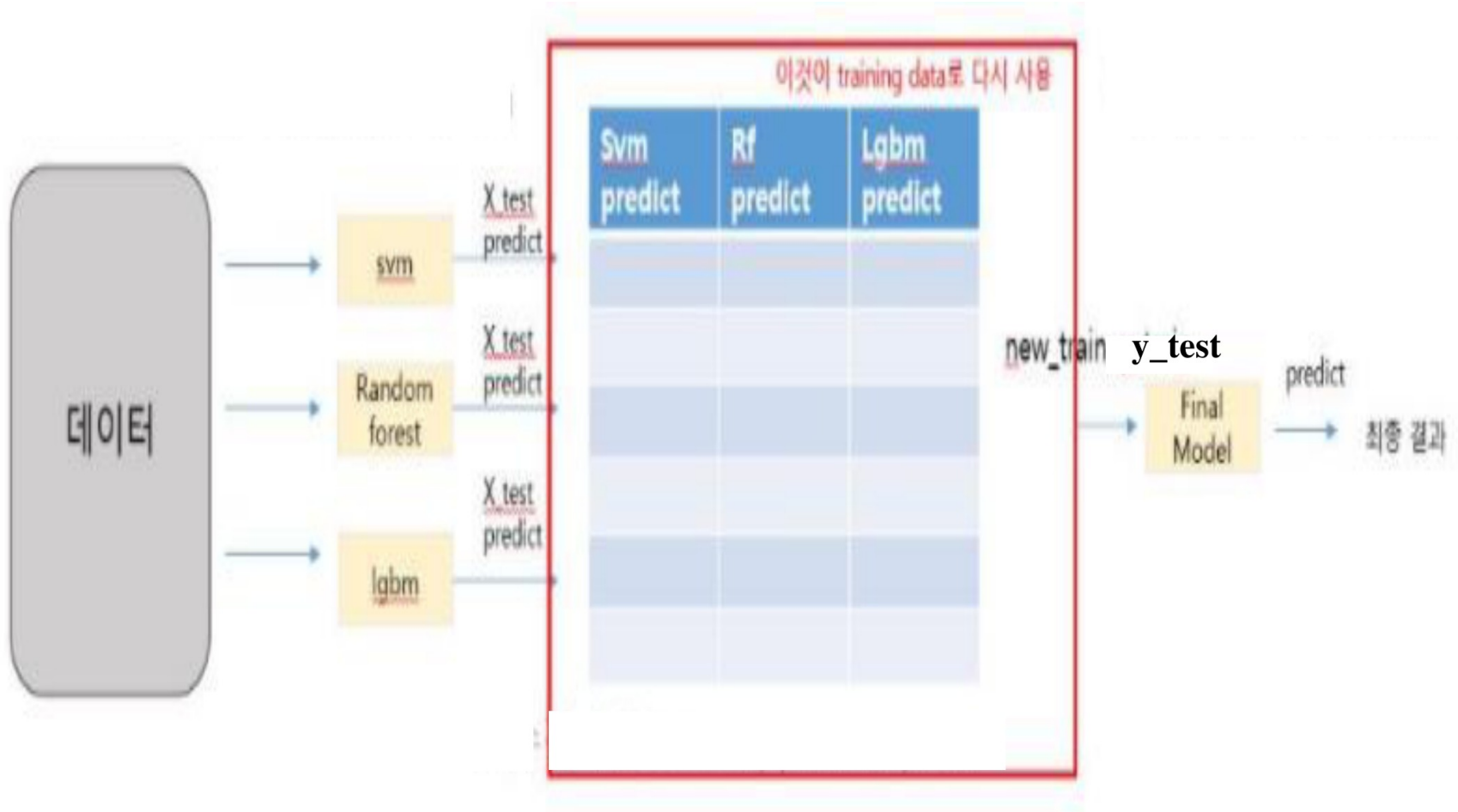
여러 모델의 예측값이 최종 모델의 입력이 된다

스태킹 앙상블 기본 구조 — SVM, 랜덤 포레스트, LightGBM 등



기본 스태킹은 테스트셋 예측값을 2단계 모델의 훈련셋으로 쓴다

기본 구조(세부) — 각 모델의 X_test 예측을 열로 쌓아 new_train을 만들고 y_test로 최종 모델을 학습한다

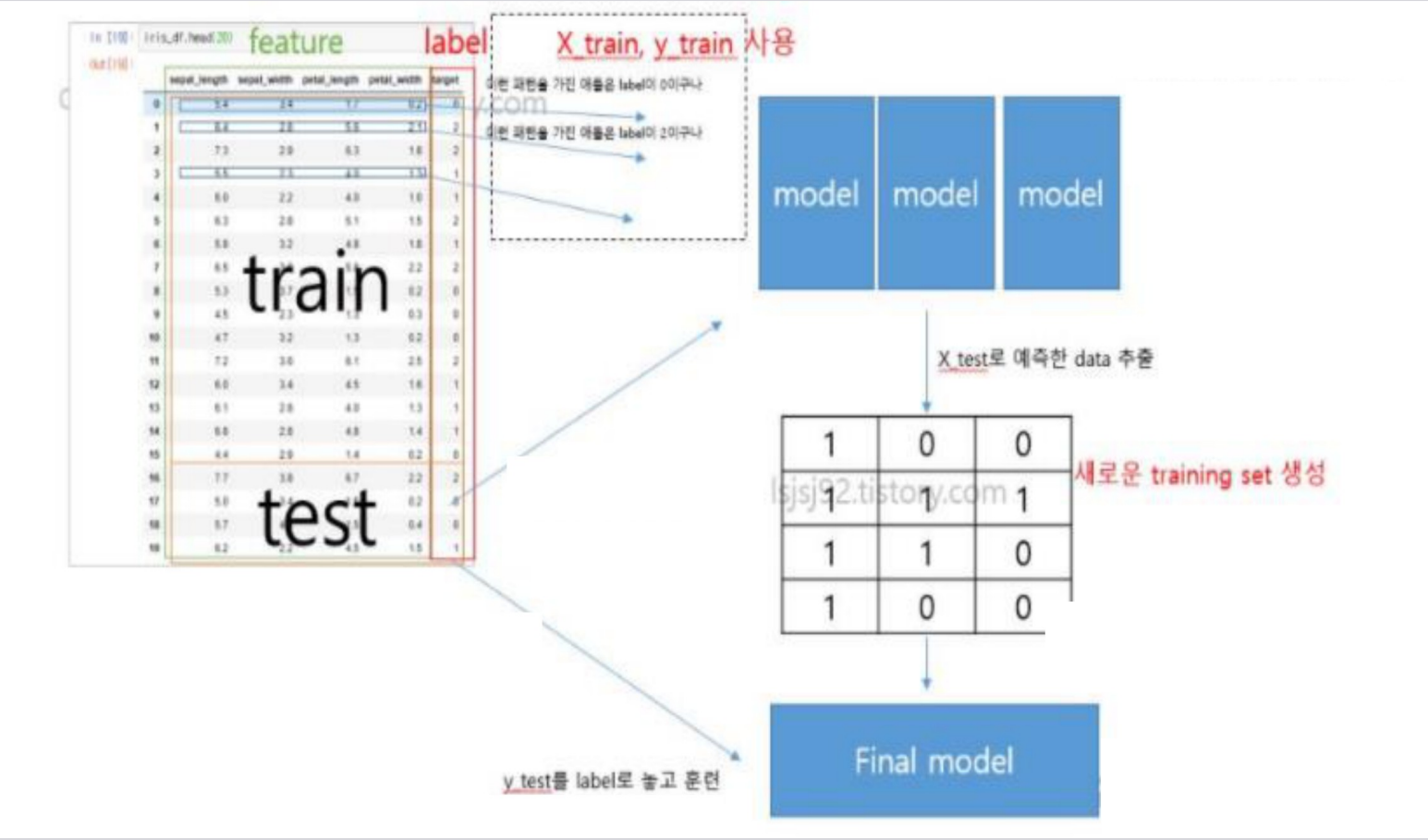


최종 모델은 기본 모델들의 예측으로 만든 새 훈련셋을 학습한다

최종 구조 — 기본 모델 학습 → X_test 예측으로 새 training set 생성 → y_test를 레이블로 훈련

주의

- 이 방식은 최종 모델이 y_test를 보고 학습한다
- 그러면 테스트셋으로 성능을 공정하게 잴 수 없다
- 다음의 CV 기반 스태킹이 이 문제를 푼다

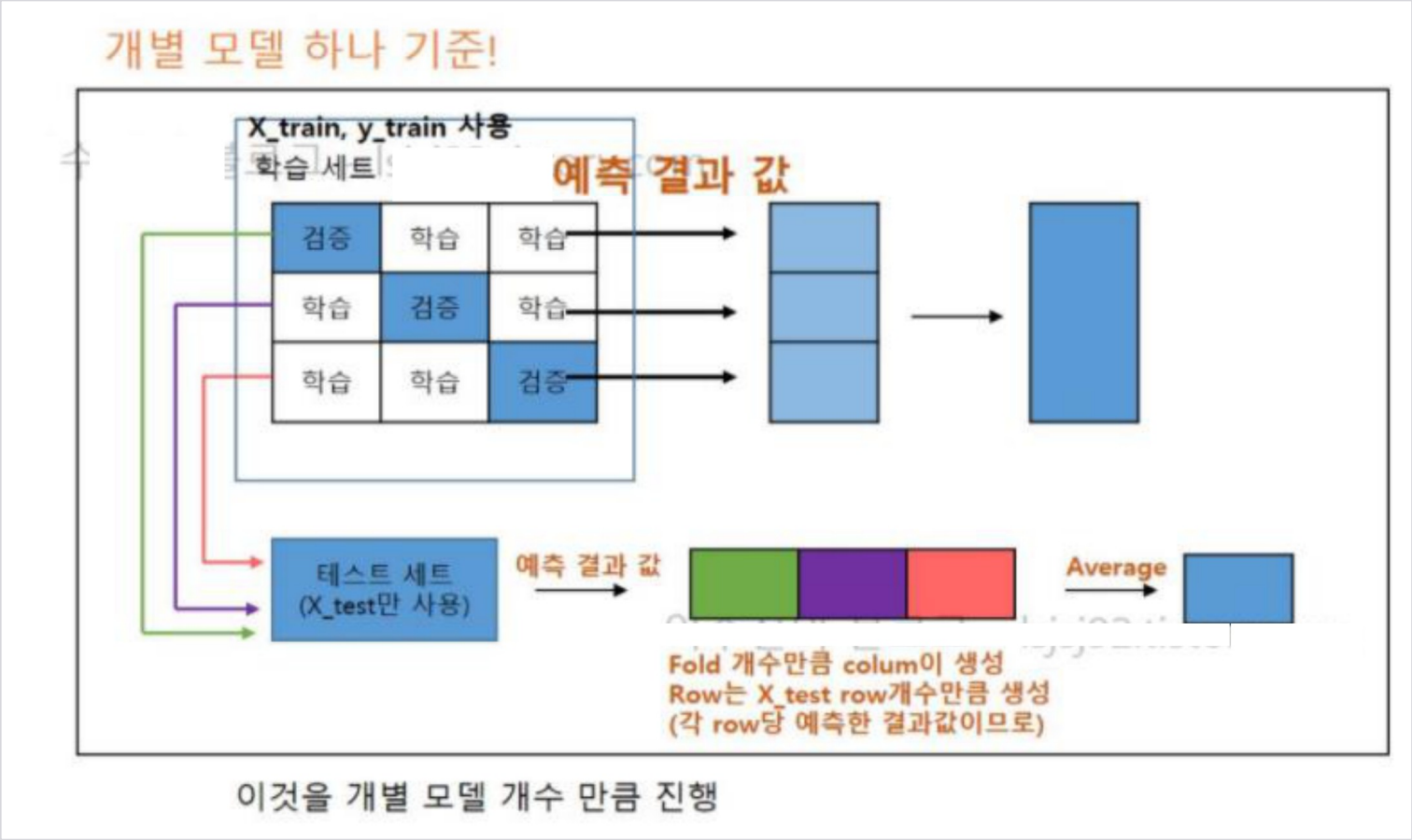


CV 기반 스태킹은 겹마다 검증 예측과 테스트 예측을 만든다

[참고] 하나의 모델로 3겹을 실행하고, 이것을 개별 모델 개수만큼 진행한다

하나의 모델 기준

- 겹마다 검증 겹을 예측해 모으면 X_train 전체의 예측 한 열이 된다
- 겹마다 X_test도 예측한다. 겹 개수만큼 열이 생기고 행은 X_test 행 수다
- 그 열들을 평균해 테스트 예측 한 열을 만든다

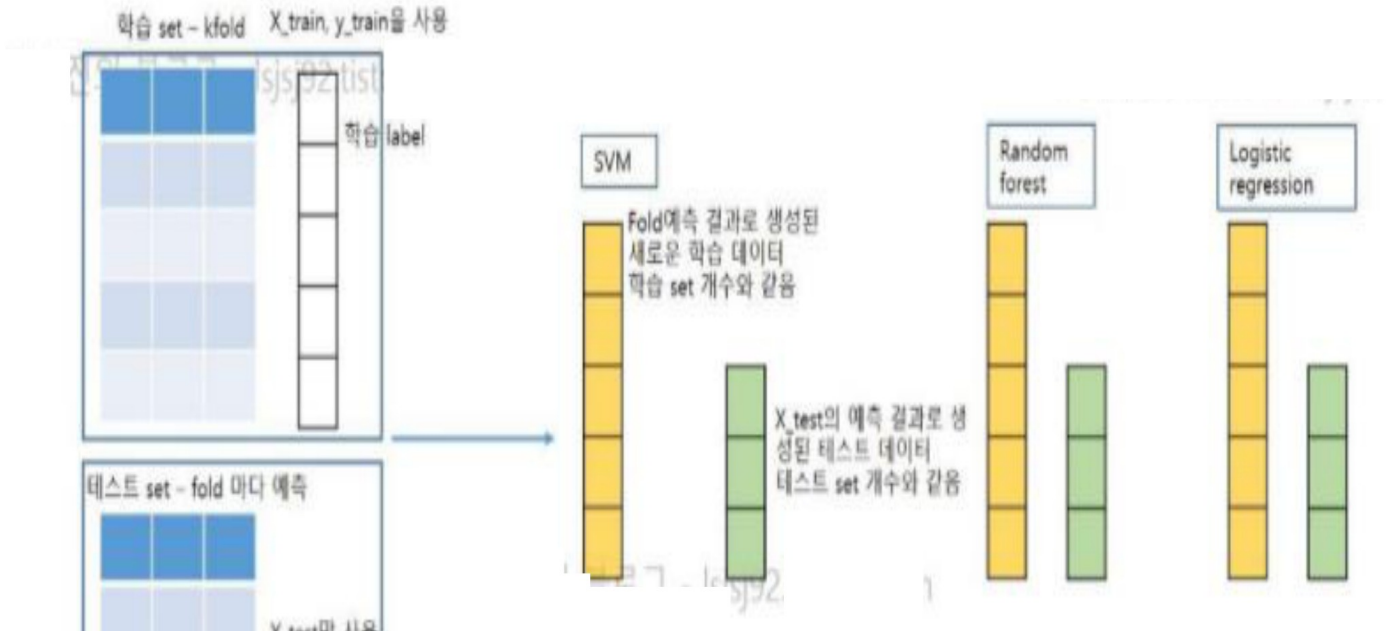


CV 예측은 훈련셋과 크기가 같아 y_train과 짝지을 수 있다

각 모델별 K겹 실행 — 노랑: 겹별 검증 예측(새 학습 데이터), 초록: X_test 예측 평균

절차

- 각 머신러닝 모델별로 KFold를 실행한다
- 겹별로 예측한 결과를 모은다 (노랑)
- 겹별로 예측한 원본 X_test의 결과를 평균해 모은다 (초록)



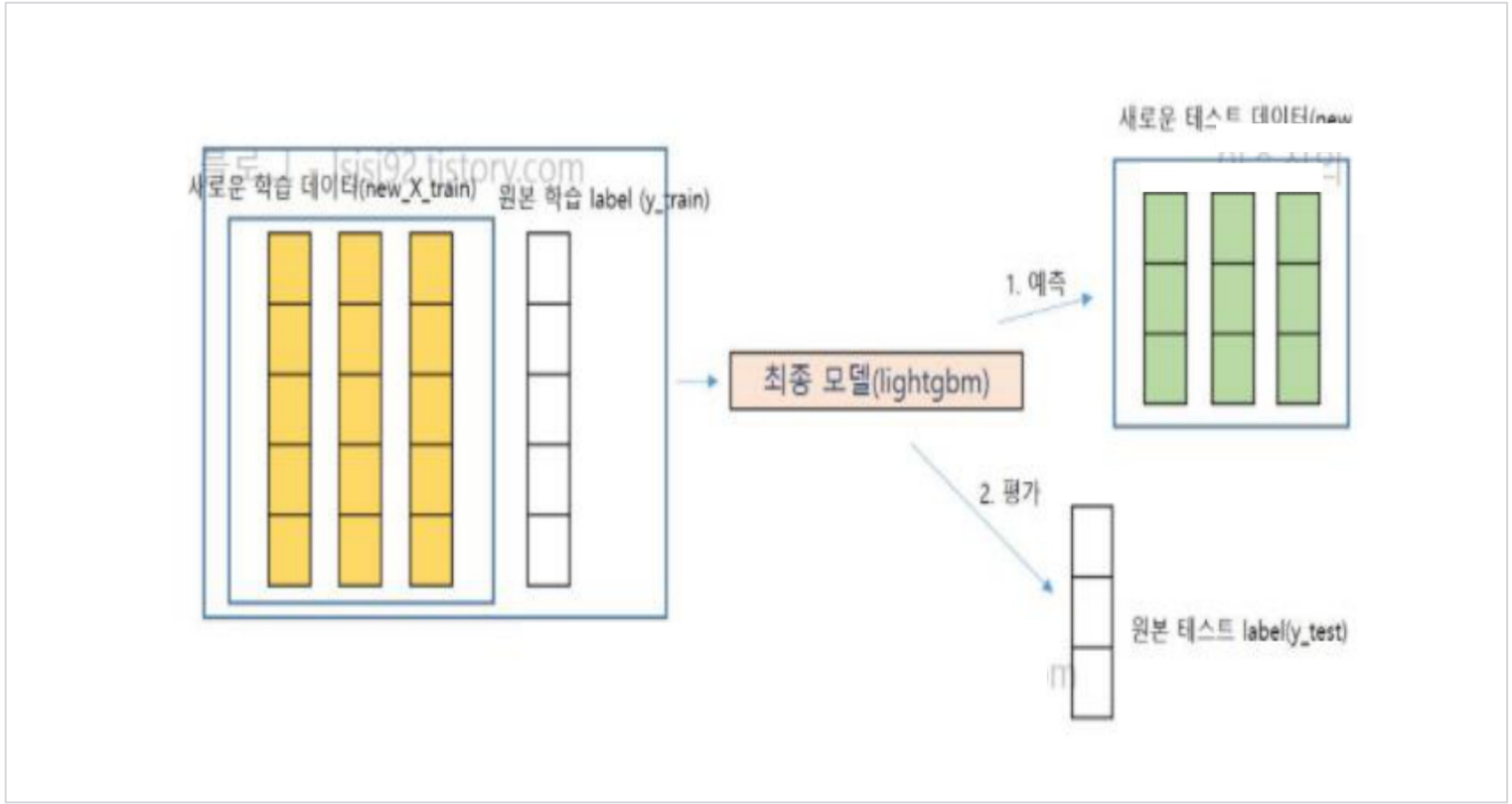
1. 각 머신러닝모델별로 KFold를 실행한다.
2. Fold별로 예측한 결과를 모은다 (노란색)
3. Fod별로 예측을 수행한 원본 X_test_set을 평균한 결과를 모은다 (초록색)

최종 모델은 y_train으로 학습하고 y_test로 평가한다

스태킹 최종 모델 — 새로운 학습 데이터(new_X_train) + 원본 학습 레이블(y_train) → 최종 모델(LightGBM)

최종 모델

- 겹 검증 예측을 new_X_train으로 모은다
- X_test 예측을 new_X_test로 모은다
- new_X_train과 y_train으로 학습한 LightGBM이 new_X_test로 최종 y_test를 예측한다



공정한 검증이 튜닝과 앙상블의 출발점이다

네 부분에서 확인한 내용

1

교차 검증

분류 문제는 층화 K겹으로 나눈다. 정렬된 데이터에서 KFold는 91.33%, 층화는 96.67%였다

2

파라미터 튜닝

그리드 → 랜덤 → 베이지안 순으로 시도를 아낀다. 획득함수는 활용과 탐험을 함께 따진다

3

앙상블과 스태킹

성능이 낮은 모델을 섞으면 앙상블이 단일 모델보다 낮아질 수 있다. 스태킹의 메타 모델은 CV 예측으로 학습해 누수를 막는다